TDMA Schedule Optimizer

Project documentation: design process, reasoning and final values for Part 1 and Part 2

Ramachandra TejSrikar Reddy · 2 October 2026

1. Summary

This project computes collision-free TDMA transmit schedules for a network of radios and loads them into an EMANE emulation.

- **Part 1** is a Python command-line tool. From the radios' coordinates it builds the radio graph, marks every pair of radios within two hops as conflicting, and colours that conflict graph so that each colour is one time slot.
- On the 16-node test set (a 4 x 4 grid with 300 m spacing and 500 m radio range) it produces a **9-slot frame with no collisions**. 9 slots is also the proven minimum for that layout.
- **Part 2** runs 16 EMANE TDMA radios in one Docker container. A bridge script converts the Part 1 schedule into EMANE's schedule XML and sends it to the radios.
- With the optimizer's schedule EMANE reported **no interference (SINR) drops**, and 99.9% of packets crossed the in-range links with 10 ms slots. A control schedule that ignores hidden terminals delivered only 34.5%.
- With the 1 ms slots the task asks for, the optimizer's schedule delivered 83.4% to 99.4% across three runs. The loss is emulator timing in a virtual machine, not collisions (section 6).

2. Problem and assumptions

In a TDMA network all radios share one frequency and take turns in fixed time slots. A frame is a repeating sequence of slots. Two radios may transmit in the same slot only if no receiver can hear both. The task defines two cases that must be avoided:

- **Direct interference (1 hop).** A and B are in range of each other. If both transmit in one slot, neither can receive the other.
- **Hidden terminal (2 hops).** A and C are out of range of each other but both in range of B. If both transmit in one slot, their signals collide at B.

Radios more than two hops apart cannot disturb a common receiver, so they may share a slot. This is spatial reuse, and it is what keeps the frame short.

Assumptions

Item	Assumption	Why
Coordinates	4 x 4 grid, 300 m spacing, numbered row by row (data/grid16.json)	The task shows only Node_01 [0,0], Node_02 [300,0] and Node_16 [900,900]. A regular 4 x 4 grid is the simplest layout consistent with all three.
Radio range	Two radios are linked if their distance is at most 500 m	Stated in the task. Configurable with --range.
Slots per radio	Each radio gets exactly one transmit slot per frame	The task asks for a node-to-slot mapping. Traffic load is not modelled.
Mobility	Positions are static	The task gives static coordinates.

3. Part 1: design process

3.1 From coordinates to a radio graph

Each radio is a node. For every pair of radios the Euclidean distance is computed, and an edge is added when it is at most the radio range. The result is the *physical graph* G: an edge means the two radios can hear each other.

3.2 Interference as a conflict graph

The two interference cases are exactly the pairs of radios that are one or two hops apart in G. Joining every such pair gives the *conflict graph*, which is the square of G (networkx.power(G, 2)). A distance-2 colouring of G is the same thing as an ordinary colouring of the conflict graph, so the scheduling problem becomes: give every node a colour that differs from all its neighbours in the conflict graph, using as few colours as possible. Each colour is one time slot.

3.3 Choosing slots with greedy colouring

Finding the true minimum number of colours is NP-hard, so the tool uses greedy colouring: visit the nodes in some order and give each one the lowest-numbered slot that none of its conflicting neighbours already holds. Greedy colouring can never produce a collision. How many slots it needs depends only on the visiting order, so the tool tries two orders and keeps the shorter schedule:

- **largest_first.** Visit nodes by number of conflicts, highest first. The order is fixed before colouring starts.
- **DSATUR.** Always pick next the node whose conflicting neighbours already hold the most different slots (the most constrained node), breaking ties by number of conflicts. The order adapts as slots are handed out.

Both orders are written out in graph_engine.py (the functions dsatur and largest_first) rather than taken from a library, so each step can be read and explained. The tests check that both produce exactly the same colourings as networkx's built-in versions; on all 1,500 random layouts of section 4.3 they do.

If both give the same frame length, the DSATUR schedule is used. Slot numbers are then relabelled in order of first use, so the first node always gets slot 0 and the output is identical on every run.

3.4 Knowing how good the answer is

A heuristic gives no guarantee, so the tool also reports a lower bound. A radio and all of its neighbours are pairwise within two hops of each other (through that radio), so each of them needs its own slot. The radio with the most neighbours therefore sets a floor of *neighbours + 1* slots. When the schedule reaches this floor it is provably optimal, and the report says so.

3.5 Independent self-check

Before printing, validator.py checks the schedule again from the raw coordinates. It does not use the conflict graph: for every pair of radios sharing a slot it confirms that they are out of range of each other and have no common neighbour. A mistake in the graph construction therefore cannot hide a collision. If a collision is found the tool prints the offending pairs and exits with an error instead of a schedule.

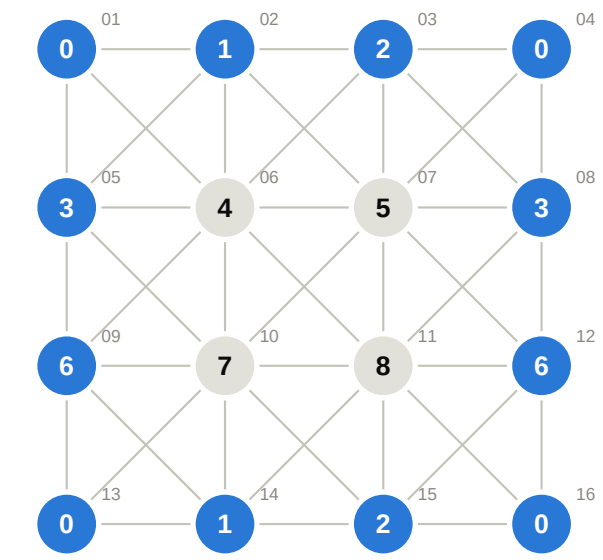
3.6 Decisions and reasons

Decision	Alternative considered	Reason
Colour the square of the physical graph	A purpose-built distance-2 colouring routine	Both give the same schedules. Reducing to ordinary colouring lets the standard colouring heuristics be applied directly.

networkx for the graph and its square; colouring written by hand	networkx's greedy_color for the colouring too	The task names networkx and asks for a colouring algorithm to be written. The hand-written colouring is checked against networkx's in the tests.
Run two heuristics and keep the best	DSATUR only	The second run is cheap and occasionally wins (section 4.3).
Report a lower bound	Report only the frame length	Shows whether the heuristic result can still be improved.
Validate from raw coordinates	Check the edges of the conflict graph	A check that reuses the conflict graph would pass even if that graph were wrong.

4. Part 1: final values

4.1 Schedule for the 16-node grid



Quantity	Value
Radios	16
Radio range	500 m
Links in the physical graph	42
Conflicting pairs (1 or 2 hops)	90
Frame length, DSATUR	9 slots
Frame length, largest_first	9 slots
Lower bound	9 slots
Collisions found by the self-check	0

Figure 1. The 16-node grid with its links. The number inside each radio is its slot; the small number beside it is the node id.

Node	Slot	Node	Slot	Node	Slot	Node	Slot
Node_01	0	Node_05	3	Node_09	6	Node_13	0
Node_02	1	Node_06	4	Node_10	7	Node_14	1
Node_03	2	Node_07	5	Node_11	8	Node_15	2
Node_04	0	Node_08	3	Node_12	6	Node_16	0

The four corner radios share slot 0, and four more slots are each shared by a pair of radios on opposite edges of the grid. The four interior radios are within two hops of every other radio, so each needs a slot of its own. That gives 1 + 4 + 4 = 9 slots for 16 radios.

4.2 Why 9 slots and not the 5 in the task's sample output

With a 500 m range each radio reaches its horizontal and vertical neighbours (300 m) and its diagonal neighbours (about 424 m). An interior radio therefore has 8 neighbours, and those 9 radios must all use

different slots. No distance-2 schedule for this grid can be shorter than 9.

The 5-slot assignment printed in the task's sample output, applied to this grid, separates direct neighbours but not hidden terminals: for example Node_01 and Node_03 both hold slot 0 while both are in range of Node_02. The sample was therefore treated as an illustration of the output format, not as a target. Two observations support this reading: the tool does produce exactly 5 slots for the same grid when the range is reduced to 350 m (no diagonal links), and section 6 shows in EMANE that the 5-slot assignment loses packets on this grid.

4.3 How the heuristics compare

On the grid both heuristics reach the optimum. To see whether they differ in general, both were run on 500 random layouts per network size, with radios placed uniformly in a 2 km x 2 km area and a 500 m range:

Radios	DSATUR shorter	largest_first shorter	Same length	Best result equals lower bound
16	3	0	497	481 of 500
30	25	1	474	439 of 500
60	142	17	341	349 of 500

DSATUR is the stronger heuristic and its advantage grows with network size, but largest_first still wins occasionally, which is why both are run. Where the best result does not equal the lower bound, the schedule was at most 3 slots above it; the bound itself is not always reachable, so those schedules may still be optimal.

4.4 Tests

The pytest suite (98 tests) covers:

- all radios out of range of each other: every radio gets slot 0;
- all radios in range of each other: one slot per radio;
- the A-B-C hidden-terminal line: three different slots;
- a 9-radio line: 3 slots, with radios three hops apart sharing a slot;
- a distance of exactly 500 m counts as in range;
- the 16-node grid: no collisions and 9 slots, with both heuristics;
- 20 random layouts per heuristic: no collisions;
- the validator reports a direct collision and a hidden-terminal collision, and rejects the task's 5-slot sample on the grid;
- command-line behaviour, including malformed input;
- DSATUR on small textbook graphs: 3 slots for a 5-cycle, 2 for a 6-cycle;
- 20 random layouts per heuristic: the hand-written colouring is identical to networkx's;
- the EMANE bridge lists exactly the right transmitters in each slot.

5. Part 2: design process

The goal of Part 2 is to show, in an emulator rather than on paper, that packets are delivered or lost according to the schedule from Part 1. The approach was to keep the EMANE side as close as possible to the model used in Part 1, so that any difference in the result can be traced to the schedule.

5.1 Emulation layout

Everything runs in one privileged Docker container. EMANE 1.5.3 is compiled from source in the image, so the same Dockerfile works on x86 and on ARM machines. Each radio lives in its own Linux network namespace and runs one EMANE instance with one NEM: EMANE's native TDMA radio model on top of the emulated physical layer, with a virtual interface (emane0) that applications send packets through. The namespaces are joined by a Linux bridge that carries EMANE's over-the-air and event multicast channels.

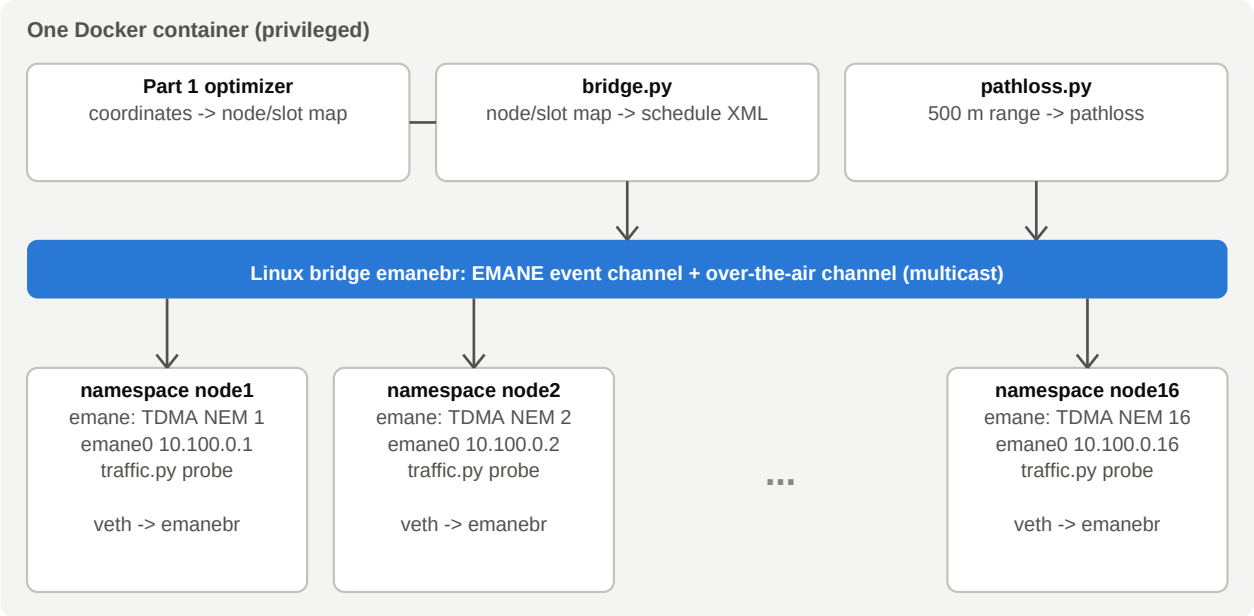


Figure 2. One container; one namespace and one EMANE TDMA radio per node. Schedule and pathloss reach the radios as EMANE events over the bridge.

5.2 Mapping the Part 1 model onto EMANE

Part 1 concept	EMANE equivalent
Node name, in input order	NEM id 1..N (Node_01 is NEM 1)
Slot (colour) held by a node	The node's NEM id is listed in that slot's element with a tx child
Frame length	slots attribute of the schedule structure; one frame per multiframe
A node is silent in every other slot	EMANE default: any slot a NEM is not given is a receive slot
Two radios within 500 m are linked	Pathloss event with the free-space loss for their distance at 2.4 GHz
Radios beyond 500 m do not interact	Pathloss event of 300 dB, far below receiver sensitivity, so the physical layer discards the signal

EMANE has no radio-range setting; what a receiver hears is decided by pathloss. pathloss.py therefore applies the same rule as Part 1 (linked if at most 500 m apart) and publishes it as pathloss events, using the precomputed propagation model.

5.3 The integration bridge

bridge.py imports the Part 1 engine, computes the schedule for the given coordinates and writes it in EMANE's TDMA schedule XML format. The start of the file generated for the grid is:

```
<emane-tdma-schedule>
  <structure frames="1" slots="9" slotoverhead="0" slotduration="1000" bandwidth="1M" />
  <multiframe frequency="2.4G" power="20" datarate="11M" class="0">
    <frame index="0">
      <slot index="0" nodes="1,4,13,16">
```

```
  <tx />
</slot>
<slot index="1" nodes="2,14">
  <tx />
</slot>
...
```

The file is sent with EMANE's own tool, `emaneevent-tdmaschedule schedule.xml -i emanebr`, which turns it into one schedule event per NEM. After sending, the demo reads each radio's scheduler.scheduleAcceptFull counter and its ScheduleInfoTable to confirm that the schedule was accepted as written.

5.4 Radio settings

Setting	Value	Reason
Slot duration	1 ms, and 10 ms for comparison	1 ms is the task's value. 10 ms separates schedule effects from emulator timing (section 6).
Frames per multiframe; slot overhead	1; 0	One repeating frame is all the schedule needs.
Frequency; bandwidth	2.4 GHz; 1 MHz	Single shared channel, as in the task.
Data rate	11 Mbit/s	A 1 ms slot carries 1,375 bytes, so a test packet fits in one slot. On EMANE's default reception curve this rate needs an SINR above 1 dB, so two overlapping signals of similar strength are lost and collisions are visible.
Transmit power; antenna gain	20 dBm; 0 dBi	Gives every in-range link an SINR far above the 8 dB needed for 100% reception when there is no interference.
Noise mode; noise figure	all; 4 dB	Every in-band transmission counts as interference in the SINR calculation.
Reception curve	EMANE default tdmabasemodelpcr.xml	No custom tuning.

5.5 Test method

All 16 radios broadcast at the same time: 300 UDP packets of 64 bytes each, 50 per second, sent through emane0. Each radio counts the packets it hears from every other radio (traffic.py). report.py then computes, for the 84 directed in-range links, the share of packets that arrived, and counts packets heard between radios that are out of range. It also sums EMANE's own MAC drop table, which records why bytes were discarded: *SINR* means lost to interference; *Slot Error* and *Long* are timing reasons, meaning the packet reached the receiver outside the slot it was sent in or too late in the slot to finish inside it.

Two schedules are compared at each slot duration: the optimizer's 9-slot distance-2 schedule, and a control that separates only direct neighbours (the 5-slot assignment from the task's sample output, emane/distance1_schedule.json).

6. Part 2: final values

Schedule	Slot	Packets delivered	Links with loss	SINR drops (bytes)	Timing drops (bytes)	Out-of-range packets
Optimizer, 9 slots	1 ms	21,009 of 25,200 (83.4%)	84 of 84	0	446,276	0

Distance-1 control, 5 slots	1 ms	16,423 of 25,200 (65.2%)	84 of 84	274,992	360,596	0
Optimizer, 9 slots	10 ms	25,168 of 25,200 (99.9%)	8 of 84	0	3,392	0
Distance-1 control, 5 slots	10 ms	8,700 of 25,200 (34.5%)	55 of 84	763,194	0	0

Table 1. Results of the recorded run (emane/out). 84 directed in-range links, 300 packets sent per radio. Timing drops are EMANE's Slot Error and Long reasons.

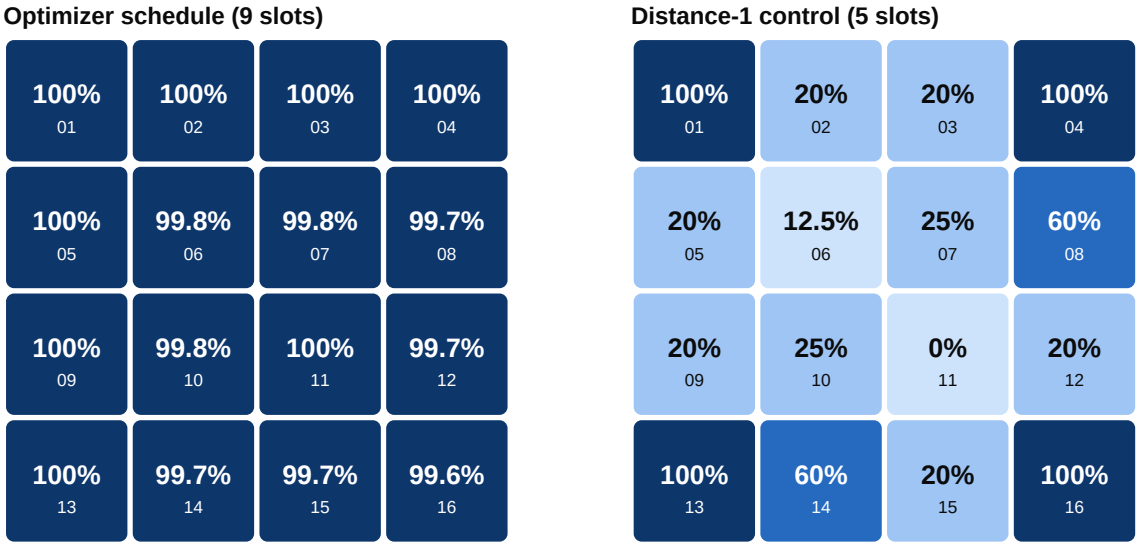


Figure 3. Share of its neighbours' packets that each radio received, with 10 ms slots. Each cell is one radio, laid out as in Figure 1.

What the results show

- **The schedule is enforced.** Every radio accepted the schedule event, and its ScheduleInfoTable shows one transmit slot and receive in all others, as generated by the bridge.
- **The optimizer's schedule is collision-free in EMANE.** EMANE recorded no SINR drops with it at either slot duration, in any of three runs. With 10 ms slots 99.9% of packets were delivered in the recorded run and 100% in the other two.
- **Ignoring hidden terminals costs most of the traffic.** The control schedule is shorter (5 slots) but delivered only 34.5% at 10 ms, in all three runs, and every lost byte was recorded as an SINR drop. 55 of the 84 links lost packets. Node_11 received 0.0% because every one of its neighbours shares a slot with another of its neighbours, while the corner radios (01, 04, 13, 16), whose three neighbours all hold different slots, still received everything.
- **The control looks better at 1 ms only because the network is less loaded.** It delivered 65.2% there. At 50 packets per second a radio uses about one in four of its 1 ms slots, so two radios that share a slot often do not transmit in the same frame. With 10 ms slots every slot is used and every shared slot collides.
- **The range model holds.** No packet was heard between radios more than 500 m apart in any run.
- **1 ms slots are at the limit of this emulation setup.** With the optimizer's schedule and 1 ms slots, 83.4% of packets were delivered in the recorded run, and all of the loss had timing reasons, none SINR. EMANE's slot statistics show most transmissions leaving 0.25 to 0.5 ms into the slot and some arriving after the slot has ended, where the model discards them. This is scheduling jitter of 16 EMANE processes sharing a 4-CPU virtual machine, not a property of the schedule, and it varies from run to run: the three consecutive runs delivered 99.4%, 88.1% and 83.4% (logs of the first two

are in emane/out/repeats).

7. Limitations and next steps

- **Coordinates are assumed.** Only three of the 16 coordinates are given in the task. The tool accepts any coordinates, so a different layout only needs a different input file.
- **Interference is modelled as a disc.** A radio beyond 500 m is treated as contributing nothing. Real radios also see the summed interference of distant transmitters; an SINR-based conflict model would capture that at the cost of a harder optimisation problem.
- **One slot per radio per frame.** Radios that carry more traffic (for example relays in the middle of the network) get the same share as the others. A natural extension is to give busy radios extra slots where the frame has room.
- **Static and centralised.** When radios move, the schedule has to be recomputed and sent again. EMANE supports this through new schedule events, so the bridge can be re-run on updated coordinates.
- **Heuristic, not exact.** When the result is above the lower bound, an exact solver (for example integer programming) could confirm or improve it for networks of this size.
- **Emulation timing.** Running EMANE on bare-metal Linux with more cores would be expected to reduce the Slot Error loss seen with 1 ms slots; this was not tested.

8. How to reproduce

Part 1 needs Python 3.10 or newer; Part 2 needs Docker.

```
# Part 1
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python src/cli.py --file data/grid16.json
.venv/bin/python -m pytest

# Part 2 (the first build compiles EMANE from source, which is slow)
docker build -t tdma-emane -f emane/Dockerfile .
docker run --rm --privileged -v "$PWD/emane/out:/work/emane/out" tdma-emane
```

Path	Contents
src/	Part 1: graph_engine.py, validator.py, formatter.py, cli.py
tests/	pytest suite
data/grid16.json	The 16-node test set
emane/	Part 2: Dockerfile, EMANE XML, bridge.py, pathloss.py, traffic.py, report.py, run_demo.sh
emane/out/	Recorded results of the EMANE runs and the demo log